

Overview of Generative adversarial network

Subject: Generative adversarial network

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

1.

Two time-scale update rule The two time-scale update rule (TTUR) is proposed to make GAN convergence more stable by making the learning rate of the generator lower than that of the discriminator. They prove that when trained this way, GANs "converge, under mild assumptions to a stationary local Nash equilibrium". They further show that this property extends to the use of the Adam optimizer, which is commonly used in stochastic gradient descent. It is important to note, however, that a local Nash equilibrium in no way signifies an absence of mode collapse - for instance, a GAN trained on MNIST collapsed to generating a single digit may satisfy the hypotheses of the paper, while still presenting mode collapse.

2.

GANs with alternative architectures The GAN game is a general framework and can be run with any reasonable parametrization of the generator G and discriminator D . In the original paper, the authors demonstrated it using multilayer perceptron networks and convolutional neural networks. Many alternative architectures have been tried. Deep convolutional GAN (DCGAN): For both generator and discriminator, uses only deep networks consisting entirely of convolution-deconvolution layers, that is, fully convolutional networks. Self-attention GAN (SAGAN): Starts with the DCGAN, then adds residually-connected standard self-attention modules to the generator and discriminator. Variational autoencoder GAN (VAEGAN): Uses a variational autoencoder (VAE) for the generator. Transformer GAN (TransGAN): Uses the pure transformer architecture for both the generator and discriminator, entirely devoid of convolution-deconvolution layers. Flow-GAN: Uses flow-based generative model for the generator, allowing efficient computation of the likelihood function.

3.

Progressive GAN Progressive GAN is a method for training GAN for large-scale image generation stably, by growing a GAN generator from small to large scale in a pyramidal fashion. Like SinGAN, it decomposes the generator as $G = G_1 \circ G_2 \circ \dots \circ G_N$, and the discriminator as $D = D_1 \circ D_2 \circ \dots \circ D_N$. During training, at first only G_N, D_N are used in a GAN game to generate 4x4 images. Then G_{N-1}, D_{N-1} are added to reach the second stage of GAN game, to generate 8x8 images, and so on, until we reach a GAN game to generate 1024x1024 images. To avoid shock between stages of the GAN game, each new layer is "blended in" (Figure 2 of the paper). For example, this is how the second stage GAN game starts:

4.

Science Iteratively reconstruct astronomical images Simulate gravitational lensing for dark matter research. Model the distribution of dark matter in a particular direction in space and to predict the gravitational lensing that will occur. Model high energy jet formation and showers through calorimeters of high-energy physics experiments. Approximate bottlenecks in computationally expensive simulations of particle physics experiments. Applications in the context of present and proposed CERN experiments have demonstrated the potential of these methods for accelerating simulation and/or improving simulation fidelity. Reconstruct velocity and scalar fields in turbulent flows. GAN-generated molecules were validated experimentally in mice.

5.

Vanishing gradient Conversely, if the discriminator learns too fast compared to the generator, then the discriminator could almost perfectly distinguish μ_{G_θ} , μ_{ref} . In such case, the generator G_θ could be stuck with a very high loss no matter which direction it changes its θ , meaning that the gradient $\nabla_{\theta} L(G_\theta, D_\zeta)$ would be close to zero. In such case, the generator cannot learn, a case of the vanishing gradient problem. Intuitively speaking, the discriminator is too good, and since the generator cannot take any small step (only small steps are considered in gradient descent) to improve its payoff, it does not even try. One important method for solving this problem is the Wasserstein GAN.

6.

(Ω_C, μ_C) , the fixed random information generator. There are 3 players in 2 teams: generator, Q, and discriminator. The generator and Q are on one team, and the discriminator on the other team. The objective function is $L(G, Q, D) = L_{\text{GAN}}(G, D) - \lambda I^\wedge(G, Q)$ where $L_{\text{GAN}}(G, D) = \mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] + \mathbb{E}_{z \sim \mu_Z} [\ln (1 - D(G(z, c)))]$ is the original GAN game objective, and $I^\wedge(G, Q) = \mathbb{E}_{z \sim \mu_Z, c \sim \mu_C} [\ln Q(c | G(z, c))] - \mathbb{E}_{z \sim \mu_Z, c \sim \mu_C} [\ln Q(c | G(z, c))]$

7.

Relation to other statistical machine learning methods GANs are implicit generative models, which means that they do not explicitly model the likelihood function nor provide a means for finding the latent variable corresponding to a given sample, unlike alternatives such as flow-based generative model.

8.

The generator's task is to approach $\mu_G \approx \mu_{\text{ref}}$, that is, to match its own output distribution as closely as possible to the reference distribution. The discriminator's task is to output a value close to 1 when the input appears to be from the reference distribution, and to output a value close to 0 when the input looks like it came from the generator distribution.

9.

Medical One of the major concerns in medical imaging is preserving patient privacy. Due to these reasons, researchers often face difficulties in obtaining medical images for their research purposes. GAN has been used for generating synthetic medical images, such as MRI and PET images to address this challenge. GAN can be used to detect glaucomatous images helping the early diagnosis which is essential to avoid partial or total loss of vision. GANs have been used to create forensic facial reconstructions of deceased historical figures.